Date : August 5, 2026
Version : 1.0

Design Variable Workflows: Pull, Push, Improve

Turn a rough product brief into a useful, tested workflow for pulling and pushing shared design data.

Timebox: roughly one working day (about 8 hours) using agentic development. You may spend more or less time; prioritize the choices that make the workflow genuinely easy to use.

1 What we want to learn

User orientation	Can you understand the day-to-day problem and make the workflow easy to discover, operate, and recover from?
Product thinking	Can you turn incomplete requirements into a coherent feature with sensible assumptions and boundaries?
Existing-code fluency	Can you work effectively in an unfamiliar repository and extend its patterns?
End-to-end ownership	Can you take a feature from rough idea through implementation, tests, and usable documentation?

2 What we are not trying to learn

You do not need to invent a design system, choose a particular technology stack, or produce a technically elegant solution for its own sake. Pragmatic, understandable decisions are more valuable than cleverness.

3 The existing product

The repository contains a small Design Variable Registry. It has an authenticated web UI backed by a central database of design-variable values. Variables are uniquely identified by ID. Users can browse variables, filter them, and edit them through the UI. The baseline also includes role-based editing, seeded example data, a read-only authenticated API example, unit tests, and Playwright E2E infrastructure. Use the repository README for local setup and credentials.

4 The user problem

Users run dozens of analysis tools locally: MATLAB, Python, CAD, FEM, and other software. These tools need the registry's design variables as inputs, and their outputs often need to update the same variables.

Today, a user must manually copy values out of the web app, run an analysis, and then update values one by one in the web app. With roughly 50 variables, three workflows, and three pull/push cycles per day, this is repetitive and error-prone.

5 Your challenge

Find and implement a more efficient workflow for pulling information from, and pushing information to, the central Design Variable Registry. The workflow may be exposed through the web app, a local tool, an API, a generated file, or a combination of approaches. Choose what best serves the users and explain why.

Your implementation should be usable by a real colleague, not only demonstrable by its author. You may add, remove, or replace placeholder data in the app as needed.

6 Working assumptions

Use these as the starting constraints. Where the brief is ambiguous, make realistic assumptions and record the major ones for review.

- About 50 people are expected to use the solution.
- This app is part of a larger set of tools. Design-variable data must remain accessible from the PostgreSQL database and remain readable and writable through the existing web app, even if you add another read/write surface.
- Where relevant, use the Linear app as the baseline for product taste and UI/UX: sleek, minimalist, and oriented toward power users, including support for useful keyboard shortcuts.
- Users work on macOS, Windows, and Linux and have different levels of technical comfort.
- Users have Codex or Claude installed, or can install them easily.
- Users use the design-variable information in dozens of tools and scripts, including MATLAB, Python, CAD, and FEM tools, and those tools produce the outputs that may be pushed back. The workflow should be agnostic to how the information is used or produced; it should not only support one tool or language.
- The company has a shared GitLab that all employees can access.
- Each user has three workflows that pull and push design variables. For example: "Update mass using Model 1," "Update cost estimates using Model 2," and "Update performance estimates using Model 3."
- Users may need to add, remove, or modify the variables used in their workflows on a weekly basis.
- For each workflow, assume that up to 50 variables may be pulled and up to 50 variables may be pushed per run. The variables going in and out do not need to be the same, and each workflow is run about three times per day.
- All users may read all design variables. Existing product permissions govern who may update them; preserve or sensibly extend this behavior.
- At minimum, the system should show who last edited a variable. You may improve the model if your workflow benefits from it.
- You may assume that two users will not update the same outputs at the same time.
- You have carte blanche to change the schema, dependencies, and repository structure, or create a separate repository for code that runs on a user's machine.
- Internet access, AI coding tools, and existing GitLab repositories are allowed.

7 Expected submission

1. **An implemented feature** that demonstrates your chosen pull/push workflow in the provided codebase or an appropriately connected companion repository.
2. **A user guide** explaining installation and everyday use for a colleague who may need help getting started.
3. **Tests appropriate to your solution.** Decide what unit, regression, integration, and/or E2E coverage gives confidence that the important workflow works. Explain notable omissions.
4. **A short assumptions and decisions note.** Record the major assumptions, trade-offs, limitations, and what you would do next with more time.
5. **A rollout perspective.** Explain how you would split up the implementation and roll it out. You do not need to ship the entire solution in one go; call out a sensible first release and future iterations or improvements you foresee.
6. **Presentation slides.** Prepare a short slide deck to present what you implemented during the technical interview, including the key workflow, notable decisions, and any important limitations or next steps.

Final note: There is no single expected architecture or "correct" interface. We are interested in the quality of your product decisions, implementation, testing judgment, and ability to make a rough requirement concrete.